

General Agents on the Web

Your first general agent, in the same browser tab where your Foundations ran. Claude Cowork on the web, nothing installed.

The AI Agent Factory by Panaversity · agentfactory.panaversity.org

Session 1 of 4

Chat versus delegated work

Three concepts, each followed by a hands-on task with a checkpoint you can see on your own screen. Have Cowork open in another tab and one low-stakes document ready. Low-stakes work only, never banking, medical, identity, or regulated data.

Hands-on 1

Chat versus Cowork

The same job in both. Watch which one keeps working after you stop typing.

Hands-on 2

The tab is a window

Close the tab, find the same task running on your phone.

Hands-on 3

A run that starts without you

Schedule a one-off task for later, close the laptop, watch it arrive on your phone.

Chat waits. Delegated work continues.

Same address, two different things

Open claude.ai today and the chat box and the agent sit at the same address. Telling them apart is the first skill of this course.

“If I stop typing, does the work stop?”

Chat box: yes. Agent surface: no. One question that separates the two at any address, from any company, this year and after.

The chat box

Waits for you every turn. It cannot start on a schedule, react to an event, or continue after you close the tab. That is what a conversation is. It is where you think, draft, and design.

The agent surface

You give it an assignment, not a message. It makes a plan, uses tools, works through steps, and produces a deliverable. It pauses only where a decision needs you, and keeps working when you stop watching.

The agent surface did not replace the chat box. It moved in next door.

Hands-on 1: chat versus Cowork

Everyone, in your own Cowork. One low-stakes document, or a public web page.

- 1** In a normal Claude chat, upload the document and ask for a summary with any deadlines. Notice: it answers, then waits for you.
- 2** Open Cowork. Give the same document as an assignment: "Read this, produce a one-page brief with deadlines as a document, and tell me when it is done."
- 3** Come back. Compare what chat gave you with what Cowork left behind.

PROMPT: Generate a sample employee salary document for an SMB with around 20 employees across three different departments. The document should also show each employee's salary payment history over the past six months.

Checkpoint

The Cowork task kept working while you were away, and a finished document exists that the chat never produced.

Notice

You started both from a text box. The difference is what the work did after you stopped typing.

If I stop typing, does the work stop? Chat: yes. Cowork: no.

The remote session: the tab is a window

A Cowork task started on the web runs on Anthropic's servers, not in your browser. Your tab, your closed laptop, and your phone are three windows onto the same session.

In your Claude account

One session, one ID

It plans, uses connectors and web tools, writes files, pauses when it needs you, and keeps going when it does not. It does not know or care whether a window is open.

➡ **Laptop, tab open**
You watch it work.

➡ **Laptop closed**
It keeps working. Nothing was running in the tab.

➡ **Phone, on the train**
Same session. You read the result or approve a step.

The one exception, shown by the instructor in the next hands-on. With Claude Desktop installed, a cloud task can borrow a folder or a browser on that computer. The borrowed tool stops when the computer is off; the session does not.

Closing the tab is not stopping the run. The session lives in your account, not in the window.

Hands-on 2: the tab is a window

Everyone, in your own Cowork. Step 3 is instructor-only unless you already run Claude Desktop.

- 1 Start a Cowork cloud task that uses only a cloud-reachable source: a Drive folder or a web page. Name the session like a work product.
- 2 Close the tab. Open claude.ai on your phone or in a second browser and find the same session, still running.
- 3 Run a harmless cowork task that on claude.ai
- 4 Ask to continue work in same session from mobile.

The run lives in your account. The tab, the laptop, and the phone are windows onto it.

Scheduled tasks: heartbeats

A schedule is the heartbeat: something other than your next chat turn starts the work. The task's shape picks the heartbeat.

Once

Run later, one time.

A reminder set for tomorrow.

On a schedule

Fire on a clock, laptop closed.

Every Monday at 8am.

Monitor

Check repeatedly. Surface a result only when a condition becomes true.

A dashboard number moves.

“A schedule is truly device-independent only if every required tool is also reachable without your device.”

A Cowork schedule runs in the cloud with connectors, skills, and plugins.

A Monday brief that needs a file on your sleeping laptop has rebuilt the laptop as a hidden dependency.

Hands-on 3: a run that starts without you

Everyone, in your own Cowork. Reuse the brief from Hands-on 1.

- 1** Ask Cowork for the same one-page brief as Hands-on 1, but scheduled: “Run this ten minutes from now and notify me when it is done.” A one-off if your plan offers it, otherwise a daily schedule you will delete after the first run.
- 2** Confirm it appears in your scheduled tasks. Then close the browser entirely, or close the laptop lid.
- 3** Optionally wait it out on your phone. Watch the notification arrive and open the session from there.

Checkpoint

A run started at a time you chose, with your browser closed, and left a result you can open.

Notice

Nothing on your computer took part. If a task needs a file that lives only on your laptop, the schedule quietly depends on the laptop again. The discipline before you schedule real work comes in session 3.

Work can start without another chat turn. That is the heartbeat.

Session 2 of 4

The surface: persistence, files, reach, gates

Four concepts, each followed by a hands-on task. You will touch memory, Projects, connectors, and approval modes on your own account, so decide now which low-stakes documents and folders you will use.

Hands-on 4

Memory and instructions

Edit a stale memory, set a Project instruction, watch a task obey it.

Hands-on 5

The three-tier audit

Produce a real deliverable and land it in your own Drive.

Hands-on 6

Connectors and scopes

Connect Drive and read exactly what it may do.

Hands-on 7

The gate lab

Manual, then Auto, then ring the gate from your pocket.

Finished work exits the platform.

The account spine: persistent context is a stack

In 2026, persistent context is not one thing called memory. Five layers persist for different reasons, over different lifetimes.

Layer	The question it answers	Typical lifetime
Session history	What happened in this particular task?	One task history
Project	What belongs to this continuing body of work?	Weeks, months, or longer
Semantic memory	What facts and preferences should carry across cloud sessions?	Until edited, reset, or disabled
Standing instructions	How should the assistant generally behave here?	Until you or an admin changes them
User-owned context file	What do I want portable, inspectable, and vendor-neutral?	As long as you maintain the file

An old session is not memory. A Project is not memory. They all persist, but for different reasons.

Hands-on 4: memory and instructions

Everyone, in your own Cowork.

- 1 Open Claude's memory settings. Read the topics it holds about you. Edit or delete one that is stale.
- 2 Create a Project for one real client or workstream. Add one instruction that applies only there, for example “updates are one page”.
- 3 Start a Cowork task inside that Project and check that it obeys the instruction. Name the task like a work product.
- 4 Say aloud which layer each thing lives in: the memory topic, the Project instruction, this task's history.

Checkpoint

One stale memory edited, one Project instruction that a task obeyed, and you can name the layer each lives in.

Notice

Local Cowork sessions do not use cloud memory. A Project instruction is not memory; it persists for a different reason.

Persistent information should live in the narrowest layer that reliably applies.

The three file tiers

The signature concept of this course. For every deliverable you make one decision: which tier does it land in?

Tier 1 • The task filesystem

Temporary working space while the session runs: drafts, scratch math, half-built files. Wiped when the task ends. Never the home of anything you need tomorrow.

Custody: theirs

Tier 2 • Platform storage

Saved to your account. Survives the task, the tab, and the week. The default landing place, in the vendor's custody, in the vendor's format, behind the vendor's login.

Custody: theirs

Tier 3 • The exit

The file leaves the platform into a system you control: a connector save to Drive, an email, a download, a local write, a repo commit. The door does not matter. The destination does.

Custody: yours

Finished work exits the platform. Everything else may stay.

Hands-on 5: the three-tier audit

Everyone, in your own Cowork. A low-stakes document already in your Drive.

- 1 In a Project, brief a small real deliverable: a one-page summary of a document already in Drive.
- 2 End the brief with one line: “List every file you created and where each one landed: temporary working space, platform storage, or a system I control.”
- 3 Let it run. Read the closing file list and name the tier of every output out loud.
- 4 Send the finished file to tier 3: a connector save to Drive or an email to yourself. Open it there, not in Cowork.

Checkpoint

The deliverable is open in your Drive, not in Cowork, and you can name the tier of every file the task listed.

Notice

The default landing place is tier 2. Nothing reaches tier 3 unless the brief asks for it. Before session 3, finish this on a real deliverable: Project 2.

Finished work exits the platform. Everything else may stay.

How an agent reaches the world

A connector is the cleanest path to a business system. When no connector exists, a browser. Full computer use needs Claude Desktop and belongs to the desktop app.

Reach path	Best for	Main advantage	Main risk or limit
Connector	Gmail, Drive, Slack, CRMs, databases with integrations	Structured permissions and actions you can read before you grant them	Scope can be too broad. Results can carry prompt injection
Browser	Portals, forms, dashboards, sites with no connector	Reaches pages no connector covers	Authenticated browsing raises action risk. Claude in Chrome inherits your logged-in session
Computer use	Apps with no connector or browser-friendly interface	Reaches the GUI directly	Highest operational risk. Needs Claude Desktop; taught in the desktop course

Two browser paths, two jobs. Claude in Chrome: “help me with this page I already have open”, in your own Chrome, with your logins. The built-in Cowork browser: “go handle that website for me”, in Claude's own separate browser, only with Claude Desktop installed and online.

Use the most structured tool that can do the job. Connector before browser. Browser before full computer use.

When the agent can both read and act

Prompt injection matters most when the agent reads content outside your trusted boundary and can take a consequential action. Both conditions, together.

The rule

Your brief is authority. External content is evidence. Instructions embedded inside external content are untrusted input, not a new boss for the agent.

Anthropic treats completing instructions found inside emails or web content as a prohibited action in every permission mode.

Six rules to carry

- ✓ Start with trusted sites and low-stakes accounts.
- ✓ Never import or reuse logins for banking, medical, identity, or other sensitive systems.
- ✓ Manual approval for unfamiliar sites, new plugins or connectors, and anything that sends, spends, deletes, publishes, or changes records.
- ✓ Give each workflow only the connectors and browser reach it needs.
- ✓ Use connectors as entry and exit: name the destination in the brief.
- ✓ Install only trusted plugins: each one widens what Claude can do and what it can reach.

Read is not send, draft is not publish, and view is not edit.

Hands-on 6: connectors and scopes

Everyone, in your own Cowork. Never a banking, medical, or identity account.

- 1 Open connectors. Connect Google Drive, or open an existing connection, and read exactly what it may do: read, create, edit, share.
- 2 If you are comfortable, do the same for Gmail. Read whether it can send, or only read and draft.
- 3 Ask a Cowork task to read one file through the connector. Then ask it to do something the scope does not allow, and watch what happens.
- 4 If Claude Desktop is installed, open the built-in browser option and notice that it asks for its own separate login state.

Checkpoint

For every connector on your account you can say whether it can read, write, or send, and you have widened nothing.

Notice

Read is not send, draft is not publish, view is not edit. Grant the smallest scope that completes the workflow.

Connector before browser. Browser before full computer use.

The gate in your pocket: three modes

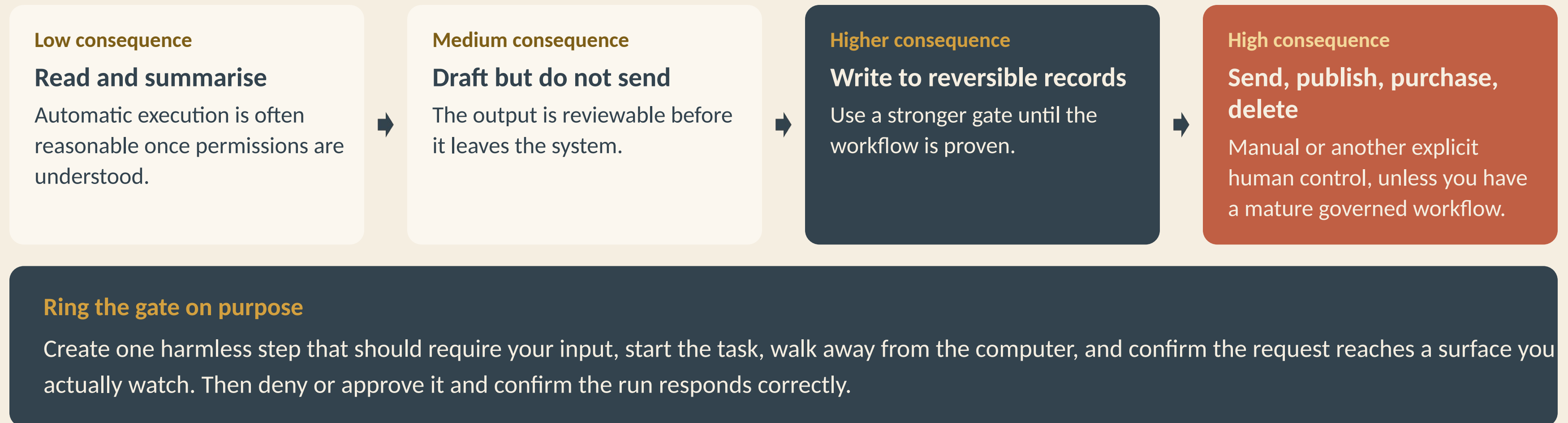
A modern surface has several gate layers at once: connector scope, organisation policy, a per-task approval mode, safety screening, and escalation to your phone.

Cowork mode	What it means	Good beginner use
Manual	Claude pauses for actions that need approval. You allow or deny them.	New workflows, unfamiliar sites or tools, sensitive or hard-to-reverse actions
Auto	Claude keeps moving and screens each action for safety, blocking or escalating ones it judges unsafe. The side panel starts here by default. The extra checks use more of your allowance.	Proven, bounded workflows where interruptions are costly but consequences stay manageable
Skip	Claude does not pause for ordinary approvals and does not apply the Auto action check.	Rarely appropriate for beginners. Only tightly trusted, bounded work

“Automatically approve” is a product default, not a recommendation. Workflow risk chooses the mode.

Match the gate to blast radius

Choose the gate deliberately before the run. Deciding autonomy when the prompt appears is designing the safety system too late.



What is allowed to happen without me? What stops or screens the action before it counts?

Hands-on 7: the gate lab

Everyone, in your own Cowork. A test folder or a draft-only mail action. Never Skip.

- 1** Pick a bounded workflow: tidy a test folder, or draft three replies with a draft-only mail action. Nothing that sends.
- 2** Run it in Manual. Count every interruption and write down what each one asked.
- 3** Run the same workflow in Auto. Note what cleared automatically and what still escalated.
- 4** Ring the gate: add one step that needs your input, start the run, put the phone in your pocket, walk away. Approve from the phone.

Checkpoint

A count of Manual interruptions, a list of what Auto cleared or escalated, and one approval that reached your phone while you were away.

Notice

Manual does not create a plan for you to approve. Auto costs more of your allowance per action. Before session 3, repeat both runs on your own bounded workflow: Project 3.

High-consequence work earns tighter gates.

Session 3 of 4

Working unwatched

The delegation loop, then the discipline before you schedule real work, each followed by a longer hands-on task. Bring the weekly chore you dislike most; you will turn it into a brief, a plan, and a schedule.

Hands-on 8

From frustration to approved plan

Let Cowork interview you, then redirect its plan with one sentence.

Hands-on 9

The first cloud schedule

Six answers, two manual walks, then attach the heartbeat to real work.

Fix the plan, not the cleanup.

The delegation loop

The product can work without you. That makes the quality of the handoff more important, not less. When the workflow is unclear, start with the frustration.

“Every Monday I spend an hour checking three places and still miss something. Interview me until you understand what I am trying to accomplish, what sources matter, what I never want changed, and what a good finished brief looks like. Then propose the workflow before doing it.”

Workflow discovery. The agent helps you find the procedure instead of waiting for you to be an automation designer.

Step 1

Brief

Outcome, audience, sources, constraints, permissions, file destinations, definition of done.



Step 2 is the leverage

Plan

Ask to see the approach before consequential work. In Cowork, put it in the brief: Manual mode does not create a plan for you.



Step 3

Approve or redirect

Fix the plan while changes are cheap. Set the permission mode to match the stakes.



Step 4

Review

Inspect the finished deliverable, the actions that mattered, and where every file landed.

Fix the plan, not the cleanup.

A brief for work you will not watch

Eight parts, in plain language. Then one line beginners routinely omit.

Outcome

What finished thing should exist?

Audience and use

Who will use it, and for what decision?

Sources

Which files, Projects, connectors, sites, or accounts may the agent use?

Boundaries

What must it not read, change, send, purchase, delete, or publish?

Method constraints

Required format, calculations, citations, date window, or business rules.

Escalation

What uncertainty should stop the run and come back to you?

Destination

Where should the final deliverable live? Name the tier-3 system explicitly.

Done

What observable conditions prove the task is complete?

The line beginners omit. “If the necessary evidence is missing, say what is missing and stop rather than inventing it.”

A confident hallucination can travel through several steps before you see it.

Review the plan, not every click

Check whether the plan is pointed at the right sources, accounts, output, and gate. A bad plan executed perfectly is still a bad run.

1

Scope

Is it touching only the work and data you named, or has the job quietly grown?

2

Order

Does it verify before it acts, or is it about to change something before the evidence is settled?

3

Reach

Is it proposing a connector, browser account, send, publish, write, or delete you did not intend?

4

Assumptions

Is it silently deciding the audience, format, business rule, or missing fact for you?

If one is wrong, redirect the plan with one sentence. For a new browser or connector workflow, stay close the first few times: watch where it reads, what it tries to write, and which approvals appear. **Trust is earned by repeated bounded runs, not assumed from the product name.**

Fixing direction before execution is cheaper than cleaning up a flawless run of the wrong plan.

Hands-on 8: from frustration to approved plan

Everyone, in your own Cowork. Pick a chore that reads and drafts, nothing that sends.

1 Paste the interview prompt with your own frustration. Answer Claude's questions honestly, including the exceptions.

2 Ask it to turn the answers into the eight-part brief, with the missing-evidence line and the file-landing line.

3 Add: "Show me your plan first and wait for my approval before taking consequential actions." Start the task in Manual.

4 Read the plan with the four checks. Redirect one thing with one sentence. Then let it run and review the output.

Checkpoint

An eight-part brief saved to tier 3, a plan you redirected with one sentence, and a first run you have reviewed.

Notice

The interview surfaces sources and exceptions you would not have written down. The redirect costs one sentence at the plan stage and an afternoon after the run.

Test the brief by hand before you schedule anything.

Six answers before you schedule

Write them down. Walk the workflow by hand at least twice. Only then attach a heartbeat. And remember that a scheduler proves the mechanism ran, not that the work succeeded.

- 1 What starts the run?** Clock, event, or watch condition.
- 2 What does it touch?** Every connector, Project, file, website, plugin, and destination.
- 3 Can it reach all of that with your devices off?** If not, move the source or accept a bridge-dependent schedule.
- 4 How will you know it succeeded?** A finished file, a dated report, a notification.
- 5 What may it do without approval?** Set the gate before the first unattended run.
- 6 What happens on the empty case?** “No new items” must be a valid result, not an invitation to invent work.

Ran: runtime evidence

The schedule fired, the agent started, the agent completed, an output exists. This proves only that the mechanism ran.

Worked: outcome evidence

The success condition was checked and the result reached the expected destination: a correctly named file in the right folder, an explicit “no new items”, a report meeting its acceptance criteria.

Scheduling photocopies a workflow into the future. It reproduces a mistake; it does not dilute it.

Session 4 of 4

Choosing, and the open path

Two concepts, each followed by a hands-on task, then the two capstone projects and the ten habits. Bring three real tasks from your coming week.

Hands-on 10

Route your own week

Three real tasks, three routes, one sentence each on the boundary crossed.

Hands-on 11

The portability test

Get one workflow out of Cowork and read it with the tab closed.

Route by what the work touches.

Web, desktop, or terminal: pick by what the work touches

Three questions: what does the work touch, must it run while my device is off, and who is allowed to hold and process the data?

Work pattern	Best starting surface	Why
One-off thinking, drafting, workflow design	Chat	No multi-step run to delegate.
Cloud files and connectors, device may be off	Cloud-only Cowork	No local dependency. Strongest unattended continuity.
Cloud work needing one approved local folder or browser	Cloud + desktop bridge	Cloud continuity, selected local tools borrowed while online.
Local files and apps, local custody matters	Local desktop agent	The machine and data are the execution surface.
A repository, terminal, tests, or CI	Coding agent or terminal	The repo and toolchain are the system of record.
A website with no connector	Browser path	Built-in browser to delegate; Claude in Chrome for the page already open.
Regulated or contractually restricted data	Stop and verify	Product availability is not permission. Written compliance answer first.

Do not choose a tool because it is newer. Choose it for where the work lives, whether it must continue without you, and who may process the data.

Hands-on 10: route your own week

Everyone, in your own Cowork. Three real tasks from your coming week.

- 1 Write down three real tasks: one on cloud documents, one on files that live only on your machine, one that needs a website with no connector.
- 2 Paste them into Cowork and ask it to route each using the three questions: what does it touch, must it run while my device is off, who may process the data?
- 3 Check its answers against the routing table. Disagree where you should.
- 4 Mark any task that touches regulated data and write “stop and verify” next to it.

Checkpoint

Three tasks, three different routes, and one sentence each on which boundary the task crosses.

Notice

A bridge is a convenience, not a custody loophole: a local file read through Claude Desktop is processed in the cloud.
Product availability is not permission.

Do not choose a tool because it is newer. Choose it for where the work lives, whether it must continue without you, and who may process the data.

The open path: no vendor cloud

Can you have this surface with no vendor cloud at all? Yes, with work. Two open paths exist today.

The managed surface

A furnished office

Sessions, files, sync, the phone gate, the scheduler: all working on day one. In their custody, in their format, at their price, under their rules.

The open path

An empty room you furnish

OpenWork connecting to a worker on infrastructure you or your organisation control. OpenCode fired by your own clock: cron on a machine you run, or a scheduled GitHub Actions job. Every part of the spine is yours to set up, run, and fix.

In exchange you get the two things no vendor sells. Custody: your data on machines you control. Choice: which model sees your prompts, and the freedom to switch. The regulated-data row of the routing table is a case where custody wins outright. A solo consultant shipping briefs by Friday is a case where the working spine wins outright.

The companies sell you a spine. The open path makes you build one.

Hands-on 11: the portability test

Everyone, in your own Cowork. Use the workflow you briefed in session 3.

- 1 Ask Cowork to write that workflow's brief, standing instructions, source locations, and definition of done into one plain-text file.
- 2 Save the file to your Drive or download it. Save the last deliverable next to it.
- 3 Close Cowork. Open the file in any text editor and read it as a stranger would.
- 4 List what you would still need if the vendor disappeared tomorrow: memory topics, Project files, connector logins, the schedule.

Checkpoint

One workflow readable with the Cowork tab closed, and a written list of what did not come with it.

Notice

The spine you rent is convenient. What you own is what travels. That list is Project 6 in miniature.

The companies sell you a spine. The open path makes you build one.

The two capstone projects

Projects 5 and 6 • after session 4.

Project 5 • One task, two harnesses

Do

Optional; needs ChatGPT Work access. Give the same low-stakes assignment to Cowork and ChatGPT Work. For each, note what starts a run, what it can reach, how it works toward the finish, what it remembers, what needs you, and where it runs. Ignore superficial button differences.

Done when

Your one-page comparison names at least one real difference in three of those six places and shows the same shape underneath.

Project 6 • The portability drill

Do

Pretend your preferred vendor disappears tomorrow. Using only tier-3 files and user-owned context, reconstruct one working workflow: instructions, source locations, definition of done, and last deliverable.

Done when

You have a written list of what you could not reconstruct.

That list is your lock-in exposure and your next portability backlog.

The habits, in one place

Ten habits. Each one maps to a concept you can now name. Your durable controls remain the brief, the plan review, the permission boundary, the checker, and the exit.

- ✓ Start with the frustration when the workflow is unclear, then let the agent interview you.
- ✓ Brief the discovered workflow in full.
- ✓ Catch problems at the plan.
- ✓ Prefer connector over browser, and browser over full computer use.
- ✓ Match the gate to blast radius.
- ✓ Send finished work to tier 3.
- ✓ Schedule only workflows you have walked successfully.
- ✓ Ask where the agent loop runs and where each tool executes.
- ✓ Read every new product by its parts: what starts it, what it reaches, what it remembers, what needs you, where it runs.
- ✓ Take the long-term runtime and custody decision to Leaving the Laptop.

Chat is where you converse. The agent surface is where you delegate.